

Optimizing HCV Genotyping: Advances in Encoding techniques and Feature Integration

Ahmed M. Fahmy^{1*}, Muhammed S. Hammad^{2†},
Walid I. Al-atabany^{3†}, Mai S. Mabrouk^{4†}

^{1*}Département d’informatique, Faculté des sciences, Université de Sherbrooke, Sherbrooke , Québec, Canada.

²Biomedical Engineering, Faculty of Engineering, Helwan University, Cairo, Egypt.

^{1,3,4*}Center for Informatics Science (CIS), School of Information Technology and Computer Science (ITCS), Nile University, 26th of July Corridor, Sheikh Zayed City, Giza, Egypt.

*Corresponding author(s). E-mail(s): studahmed91@gmail.com;

Contributing authors: muhammedsayed@h-eng.helwan.edu.eg;

W.Al-Atabany@nu.edu.eg; masaid@nu.edu.eg;

[†]These authors contributed equally to this work.

Abstract

Genomic sequence classification is a critical task in understanding the genetic makeup and variability of viruses, such as the Hepatitis C Virus. In this study, we explore the effectiveness of advanced encoding techniques, supplementary features, and oversampling methods for enhancing the accuracy of Hepatitis C Virus genotyping. Four encoding methods—Frequency Chaos Game Representation (FCGR), k-mer encoding, One-hot encoding, and Label encoding—were evaluated across multiple machine learning models, including XGBoost, Random Forest, Multi-Layer Perceptron (MLP), and K-Nearest Neighbors (KNN). To address class imbalance, Synthetic Minority Oversampling Technique (SMOTE) was applied, along with the integration of supplementary features, namely GC skew and GC content. Our results show that the FCGR and kmer encoding consistently outperform the one-hot and label encoding, with XGBoost achieving the highest overall accuracy of 99.29%. The inclusion of supplementary features further improved model performance, particularly for FCGR and k-mer encoding, by providing additional sequence-level

compositional insights. SMOTE significantly enhanced classification accuracy, especially for FCGR and k-mer encoding, highlighting the importance of addressing data imbalance in genomic analysis. This study highlights the importance of combining different encoding techniques, supplementary features, and class balancing strategies to achieve high accuracy in Hepatitis C Virus genotyping. Our findings provide a foundation for further research in applying machine learning to sequence-based classification tasks in virology and genomics.

1 Introduction and Background

Hepatitis C Virus (HCV) remains a major global health concern, affecting approximately 71.1 million individuals worldwide, primarily transmitted through direct blood exposure [1]. It is a leading cause of serious liver diseases, including cirrhosis and liver cancer, with infections manifesting as either acute or chronic conditions [2]. HCV is genetically diverse, comprising seven major genotypes and 67 subtypes, with around 30% variability between genotypes and 15% between subtypes [3].

The development of direct-acting antivirals (DAAs) has significantly improved treatment outcomes, achieving sustained virological response (SVR) rates over 90% in 12 weeks or less. However, accurate genotyping remains essential for guiding DAA-based therapies and optimizing clinical outcomes [4, 5]. Moreover, it plays a role in epidemiological surveillance, tracking genotype-specific transmission patterns, and detecting emerging drug-resistant variants, thereby contributing to effective public health [3, 6]

Despite its importance, HCV genotyping is challenging due to high genetic variability caused by rapid mutation and evolution [7]. Class imbalance is another challenge often encountered in genomic datasets, especially in HCV genotyping, and can lead to biased predictions. Moreover, traditional genotyping methods face limitations, such as difficulty in accurate identification and vulnerability to contamination [8, 9].

Currently, a range of HCV genotyping tests, such as PCR-RFLP and PCR with genotype-specific primers, are commonly used in clinical settings but suffer from notable limitations in accurately identifying certain genotypes. Studies have shown that methods like 5'NCR PCR-RFLP and core region type-specific PCR frequently misclassify genotype 6, with accuracy rates as low as 14% and 57.1%, respectively [8]. Additionally, RT-PCR-based techniques are prone to false positives and negatives, compromising their reliability [10, 11]. Misclassification issues and challenges

like cross-reactivity and contamination from previously amplified material further undermine the accuracy of these approaches [9, 12].

To overcome the limitations of traditional clinical methods, Recent research has turned to computational methods, especially machine learning, as scalable, accurate, and cost-effective alternatives to traditional clinical techniques for virus identification. These approaches reduce reliance on complex laboratory procedures while improving both speed and precision in analyzing large datasets.

Recent HCV identification studies fall into two main categories: clinical data and genomic data. While most research has focused on clinical data from lab results and health records, a growing number of studies are increasingly utilizing viral genomic sequences to investigate HCV.

Many studies have utilized machine learning algorithms such as Random Forest (RF), Support Vector Machine (SVM), Logistic Regression (LR), and k-nearest neighbors (KNN) for HCV detection. RF often outperformed other models, demonstrating high accuracy across various studies [13, 14, 15]. Advanced ensemble techniques, including hybrid models that combine RF and SVM [15], improved prediction performance. From another perspective, to handle imbalanced data, SMOTE was commonly applied to handle class imbalance, enhancing model robustness [14, 16, 17, 18, 19]. Feature selection methods such as Sequential Forward Selection (SFS) [20] and univariate statistical tests [21] also contributed to performance improvements. LR, SVM, and XGBoost were also effective in classifying patients based on blood test data [22].

Qiu et al. [23] developed a rapid and accurate method leveraging a Position Weight Matrix (PWM) to identify genotype-specific nucleotide signatures across regions of the HCV genome, including 5'NCR, CORE, E1, and NS5B. Using SVM and RF models trained on sequences from NCBI, their results showed superior performance for features derived from NS5B and E1 regions compared to CORE and 5'NCR. However, their approach required selecting specific genomic regions, limiting its ability to capture genetic variations outside targeted areas. Furthermore, the study used a smaller dataset of 10,014 samples and excluded partial sequences missing more than one-third of selected positions, potentially limiting generalizability. In contrast, more comprehensive methods that utilize entire or partial sequences without region selection and larger datasets, such as 51,092 samples, can provide more robust and accurate genotyping.

Xie et al. [24] proposed CAP-m7G, a capsule network-based model that utilizes Chaos Game Representation and reconstruction layers to identify RNA N7-methylguanosine sites with high accuracy, outperforming previous methods across multiple performance metrics. In addition, Yao et al. [25] introduced a hybrid approach that integrates attention-based mechanisms with deep learning for biomedical sequence classification, emphasizing interpretability and feature-level granularity.

Fahmy et al. [26] proposed a deep learning framework combining Chaos Game Representation (CGR) with self-supervised convolutional autoencoders to classify viral

genotypes, achieving over 99% accuracy across ten classes. However, the approach suffers from limited interpretability and high computational cost. Similarly, Ahmed et al. [27] introduced an LSTM-based model for classifying HCV proteins into structural/non-structural categories and detailed subtypes, attaining accuracies of 99.7% and 99.6%, respectively. Despite its strong performance, the model’s generalizability is constrained by the relatively small dataset, as the current dataset size remains a limiting factor.

This study systematically examines the effectiveness of advanced encoding strategies, supplementary features, and oversampling techniques for HCV genotyping. It evaluates four encoding methods—FCGR, k-mer, One-hot, and Label encoding—applied with and without additional genomic features like GC content and GC skew across various machine learning models (XGBoost, Random Forest, MLP, and KNN). The study also analyzes the impact of SMOTE on addressing class imbalance and improving classification accuracy.

Unlike previous work, which often focuses on a single encoding method or ignores the role of sequence-level supplementary features, our study uniquely integrates encoding techniques with biologically meaningful features and oversampling. This combination improves classification performance and enhances interpretability by linking statistical features to known genomic properties. Additionally, we demonstrate that our approach outperforms traditional setups on imbalanced datasets and provides consistent performance even when applied to partial sequences, making it highly applicable in practical diagnostic pipelines.

The key contributions of this study are:

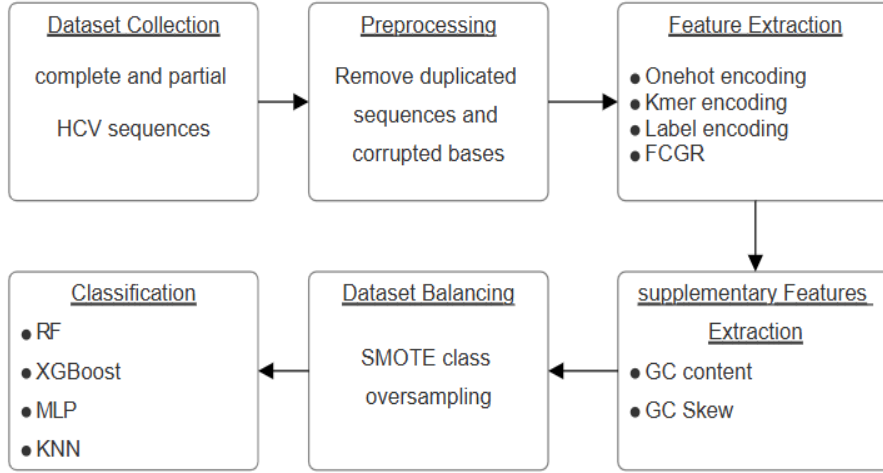
1. Proposing an optimized and interpretable framework for accurate HCV genotyping that works for both partial and complete sequences.
2. Systematically addressing class imbalance through the integration of SMOTE and evaluating its effect across different encodings.
3. Evaluating and comparing the performance of four distinct encoding techniques combined with supplementary features to establish, biologically-informed sequence classification.

The remainder of this paper is organized as follows:

Section 2 details the research methodology, covering key aspects such as the dataset used, comparative analysis of the different genotypes, oversampling techniques to address class imbalances, encoding methods for feature representation, and the incorporation of supplementary features to enhance classification performance. Section 3 presents the results and discussion, analyzing the performance of the proposed approach and comparing it with other machine learning models. Finally, Section 4 concludes the paper and summarizing key findings.

Table 1: Distribution of Complete and Partial Sequences by Genotype

HCV Genotype	Number of Complete Sequences	Number of Partial Sequences
1a	7	8627
1b	32	8112
2a	21	2044
2b	7	1906
2c	3	893
3a	4	8216
3b	0	1446
4	8	5522
5	0	1295
6	5	5214

**Fig. 1:** The proposed approach pipeline

2 Research Methodology

This section outlines the systematic approach employed in this study to evaluate various encoding methods and preprocessing techniques for genotyping HCV. It details the data collection process, comparative analysis, encoding strategies, supplementary feature extraction and the oversampling methods. We illustrate the employment of multiple encoding techniques, such as k-mer, one-hot, FCGR (Frequency Chaos Game Representation) and label encoding, alongside additional sequence features like GC content and GC skew. In this section, we propose comprehensive analysis for addressing challenges like class imbalance and sequence representation. Figure 1 shows the pipeline of the proposed approach.

2.1 Dataset

In this work, genomic sequences were obtained from Los Alamos Hepatitis C Sequence Database [28, 29]. The dataset consists of a collection of both partial

and complete sequences, covering 10 HCV genotypes: 1a, 1b, 2a, 2b, 2c, 3a, 3b, 4, 5, and 6. Table 1 visualizes the distribution of complete and partial sequences in the dataset. The dataset includes a total of 87 complete sequences and 43,275 partial sequences. Notably, genotypes 1a and 1b are the most represented, with 8,627 and 8,112 partial sequences, respectively, while genotypes 2c and 5 have the fewest complete sequences, with only 3 and 0, respectively.

2.2 Comparative Analysis

Comparative analysis in bioinformatics plays a pivotal role in understanding viruses' genetic makeup, evolutionary dynamics, and functional characteristics. In virology, viruses exhibit a remarkable diversity in their genetic sequences, even within the same species or strain. Comparative analysis involves the systematic comparison of these sequences, shedding light on the similarities, differences, and underlying patterns among viral genomes. This approach allows researchers to discern conserved regions crucial for the virus's survival, identify genetic variations influencing virulence, and trace the evolutionary history of viral strains. Following Ahmed and Jeon [30], we conducted a comprehensive data analysis employing diverse interpretation and visualization methods to help unveil hidden details within the genomes of various HCV genotypes. This includes exploring the genome sequence lengths, visualizing nucleotide distributions, examining DNA nucleotide frequencies, analyzing tri-nucleotide composition, and assessing GC percentages to identify genomic stability.

Figure 2 shows the composition of each nucleotide in the genome of each genotype, revealing higher ratios of nucleotides C and G compared to nucleotides A and T across all HCV genotypes. Furthermore, a consistent similarity in the ratios of nucleotides C and G is observed across all genotypes. Variations in the amount of Adenine–Thymine (A-T) and Cytosine–Guanine (C-G) are evident in genotypes 4, 2c, 6, 1b, 1a, and 3b. Using this information, we can determine the GC content in each genotype, which provides insights into the stability of the DNA. The GC-content, also known as guanine-cytosine content, represents the proportion of guanine (G) and cytosine (C) bases out of a total of four nitrogenous bases, which also includes adenine and thymine in DNA and adenine and uracil in RNA. DNA with low GC content is less stable than DNA with high GC content. Our findings indicate that genotypes 1a and 1b exhibit the highest CG-content at 60%, signifying their superior stability within our dataset, followed by genotype 3b of 59% and genotypes 4, 2c of 58% and 2a, 5 and 6 which are 57% and 56% for genotype 2b which is considered the least stable among all the genotypes.

The tri-nucleotide composition of each HCV genotype was also analyzed. Tri-nucleotide has great importance in bioinformatics and computational biology. Figure 3 shows the top 10 tri-nucleotides of the ten genotypes. The importance of tri-nucleotides in bioinformatics lies in their role in coding for amino acids during protein synthesis. Each set of three nucleotides, known as a codon, corresponds to a specific amino acid or serves as a start or stop signal in the genetic code. Additionally, tri-nucleotides have significance in various aspects of genomic analysis, including Identification of species and Antiviral Drug Development.

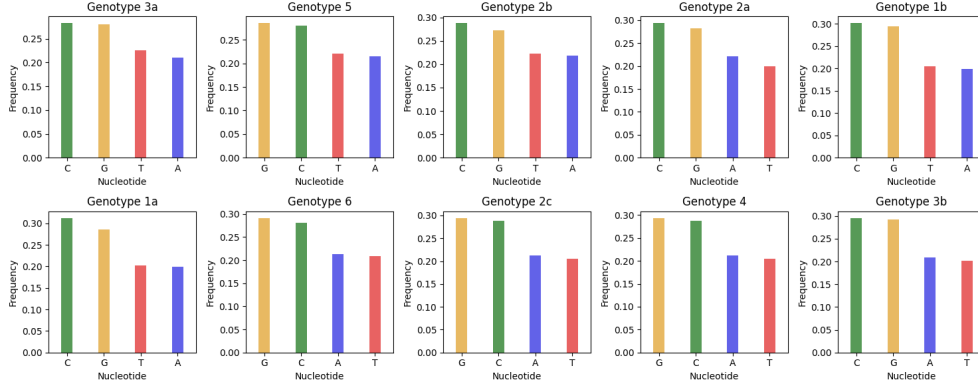


Fig. 2: Nucleotide ratios of the DNA sequence of the 10 genotypes of HCV

Table 2: Sample Distribution Before and After SMOTE

Genotype	Before SMOTE	After SMOTE
1a	6,907	6,907
1b	6,515	6,907
2a	1,652	6,907
2b	1,530	6,907
2c	717	6,907
3a	6,576	6,907
3b	1,157	6,907
4	4,424	6,907
5	1,036	6,907
6	4,175	6,907

2.3 Class Oversampling

Class imbalance is a common issue in genomics data and poses a significant challenge to building robust machine learning models. In this study, the dataset exhibited a highly imbalanced distribution of samples across genotypes, as shown in Table 2. Genotypes 1a (8,634), 3a (8,220), and 1b (8,144) had the highest representation, whereas minority classes such as 2c (896), 5 (1,295), and 3b (1,446) had significantly fewer samples. This imbalance can lead to biased predictions, where the model favors majority classes and performs poorly on underrepresented genotypes. To address this issue, we applied SMOTE, a technique that generates synthetic samples for minority classes by interpolating between existing samples in feature space, preserving the relationships between features. After applying SMOTE, the dataset was balanced with 6907 samples for each genotype, as shown in Table 2. This balancing ensured that the model could learn effectively from both majority and minority classes, avoiding the bias introduced by the original distribution. To

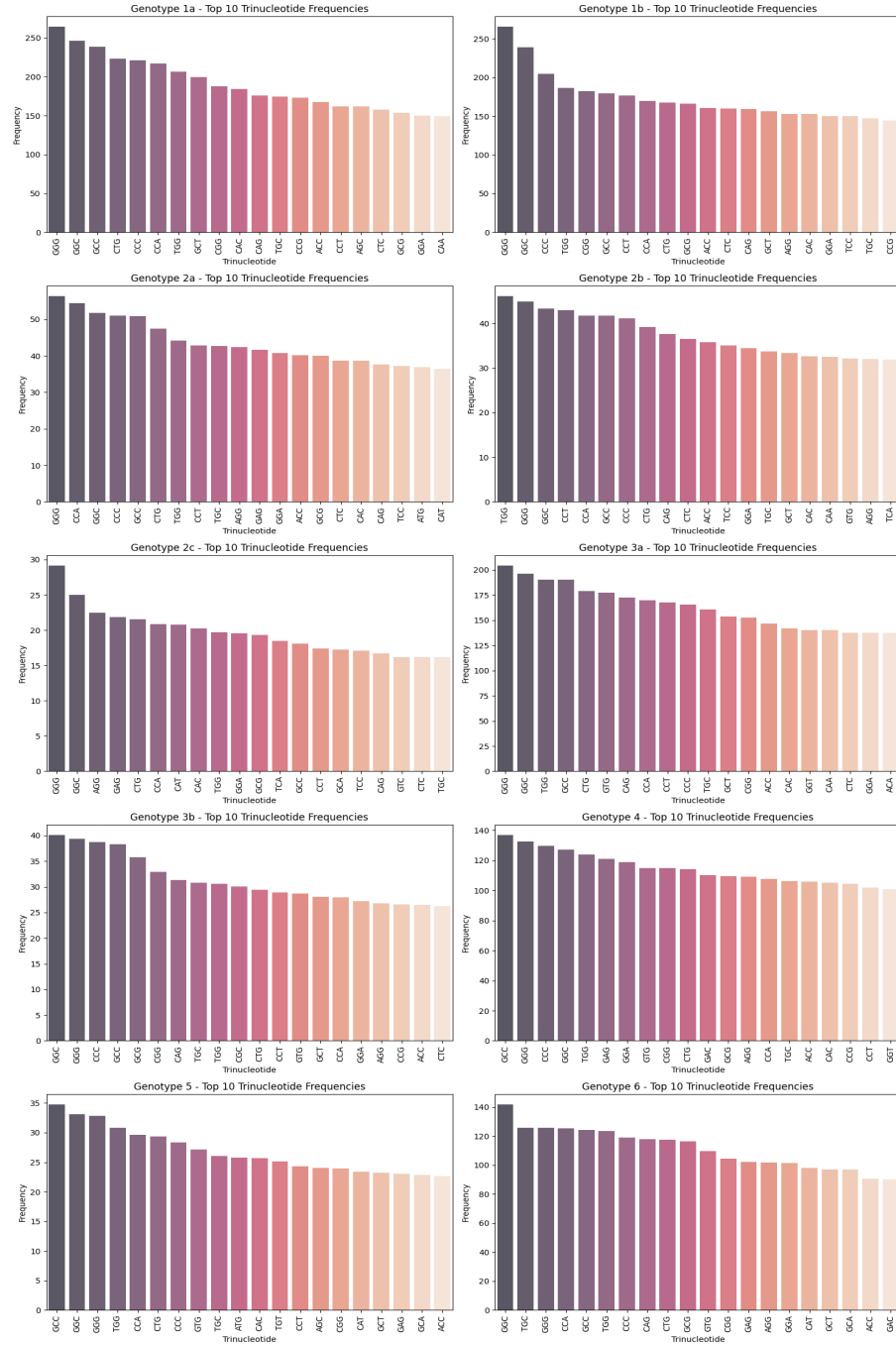


Fig. 3: tri-nucleotide distribution of the HCV genotypes

ensure unbiased evaluation, SMOTE was applied only to the training set, while the test set remained untouched to maintain its original class distribution.

2.4 Encoding Methods

In this section, we explore the various encoding techniques employed to represent the data for analysis. The choice of encoding method plays a critical role in transforming raw input data into a format suitable for computational models, thereby influencing the performance. In this study, four distinct encoding methods were utilized: k-mer encoding, label encoding, one-hot encoding, and FCGR. Each of these techniques offers unique advantages depending on the nature of the data and the specific requirements of the model.

2.4.1 Label Encoding

Label encoding is a basic numerical representation method to process DNA sequences of the target virus. This technique assigns a unique integer to each nucleotide, enabling computational analysis with algorithms that require numerical inputs. Unlike one-hot encoding, label encoding produces a more compact representation by avoiding the high dimensionality of binary vectors. Each nucleotide in the DNA sequence—adenine (A), thymine (T), guanine (G), and cytosine (C)—is assigned a distinct integer:

$$A = 0, C = 1, G = 2, T = 3.$$

For example, the DNA sequence "ATGC" is encoded as:

$$[0, 3, 2, 1].$$

Label encoding offers several advantages in sequence analysis. First, it is computationally efficient due to its low-dimensional representation, which reduces memory and storage requirements compared to one-hot encoding. Second, it is straightforward to implement and integrates well with various machine learning algorithms.

The label encoding workflow consisted of the following steps: (1) Integer Mapping: where each nucleotide in the sequence was assigned a unique integer value. (2) Sequence Adjustment: to handle sequences of varying lengths, shorter sequences were padded, while longer sequences were truncated to a predefined fixed length. This step ensured uniformity in input dimensions for machine learning models. However, the limitation of the label encoding is the implicit ordering of nucleotides. Assigning integers to nucleotides creates a numerical hierarchy that does not exist biologically. For instance, the difference between "A" and "T" (encoded as 0 – 3) is numerically equal to that between "C" and "G" (encoded as 1 – 2), which might introduce unintended biases in some machine learning models. Although label encoding is less computationally demanding, it is most effective in scenarios where the order of the labels does not influence the learning process. For more complex analyses, complementary techniques such as one-hot encoding or embedding representations may provide better performance by avoiding numerical biases.

2.4.2 One-Hot Encoding

One-hot encoding is a numerical representation method to process DNA sequences of the target virus for computational analysis. This approach is widely used in bioinformatics to transform DNA sequences into binary matrices, enabling compatibility with machine learning and statistical models. The four nucleotides are each represented as mutually exclusive binary vectors:

$$A = [1, 0, 0, 0], C = [0, 1, 0, 0], G = [0, 0, 1, 0], T = [0, 0, 0, 1].$$

For example, the DNA sequence "ATGC" is encoded as:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

One-hot encoding offers several advantages. It preserves the original information without imposing an artificial order on nucleotides, avoiding unintended implications about similarity or hierarchy among bases. The method integrates seamlessly with neural networks and machine learning algorithms, while also being flexible enough to accommodate sequences of varying lengths through padding or truncation. The one-hot encoding workflow consisted of the following steps: (1) mapping each nucleotide to its binary vector representation, (2) standardizing the data by padding shorter sequences with zeros or truncate longer sequences to create uniform input matrices. This approach was particularly effective for viral genotyping, as it preserved both positional and compositional information of the sequences. While one-hot encoding has many strengths, it can result in high-dimensional representations, particularly for long sequences. This issue increases computational requirements, but can be mitigated using techniques such as dimensionality reduction or embedding representations.

2.4.3 K-mer Encoding

K-mer encoding is a robust technique widely applied in bioinformatics to transform nucleotide sequences into numerical features suitable for computational analysis. K-mers are substrings of length k obtained by sliding a window of size k across the sequence. For example, a sequence $S = \text{"ATGCGT"}$ with $k = 3$ generates the K-mers $\{\text{ATG, TGC, GCG, CGT}\}$. This representation captures genomic features crucial for identifying genotypic variations [31]. Figure 4 illustrates the process of DNA sequence encoding using kmer technique. To balance granularity and computational efficiency, $k = 6$ was selected based on empirical testing, with smaller k values capturing broad patterns and larger values providing more detailed insights. K-mer encoding facilitates capturing sequence-specific features, enabling effective machine learning applications in viral genotyping. Its scalability and compatibility with computational pipelines make it particularly suited for large-scale genomic studies.

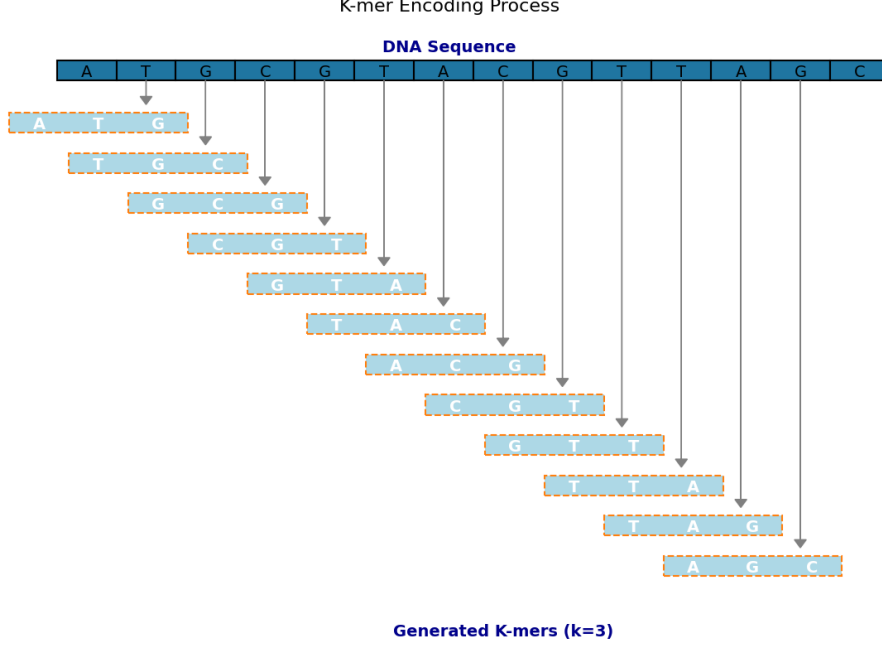


Fig. 4: Sequence encoding using the K-mer technique

2.4.4 FCGR Encoding

FCGR was employed to encode DNA sequences into a numerical format that preserves both compositional and spatial information. This technique transforms DNA sequences into a two-dimensional grid, offering a compact and informative representation of k-mer distributions. By combining the frequency and spatial arrangement of k-mers, FCGR provides a rich dataset that enables effective feature extraction for downstream machine learning applications, such as classification and clustering of genomic data. To construct the FCGR, we used different values of k , where each k-mer (a substring of length k) in the DNA sequence was mapped to unique coordinates on the grid. The mapping process involved recursively determining the position of each k-mer based on the nucleotide composition. Each nucleotide (A , T , C , G) was assigned a specific coordinate pair, which contributed to determining the position of the k-mer within the grid. For example, A and T were mapped to $(0, 0)$ and $(1, 0)$, respectively, while C and G were mapped to $(0, 1)$ and $(1, 1)$. By iterating over the k-mers in the sequence, their contributions to the grid were accumulated, producing a frequency map that effectively captured the compositional patterns of k-mers throughout the sequence.

For example, to construct the first-order FCGR matrix ($k = 1$) for a given sequence s , we start by calculating the frequency of each nucleotide (A , T , C , and G) in the sequence. This involves counting the occurrences of each nucleotide and then using

these frequencies to construct a 2×2 matrix, where each cell represents the frequency of a specific nucleotide. The first-order FCGR matrix ($k = 1$) is given by the following equation:

$$\text{FCGR}_1(s) = \begin{bmatrix} f_C & f_G \\ f_A & f_T \end{bmatrix},$$

where f_i denotes the frequency of nucleotide i in the sequence. For instance, f_A represents the frequency of nucleotide A , while f_T represents the frequency of nucleotide T . This simple matrix captures the overall nucleotide composition of the sequence, providing a basic but informative representation of its composition. To extract features from the FCGR, the grid was divided into equal-sized sub-regions. Each sub-region corresponded to a portion of the sequence space, and the frequencies were calculated as the sum of k-mer counts within the region. Using sub-grid sizes of 8×8 (selected based on grid search), the entire grid was partitioned into 64 sub-regions, and a feature vector of 64 values was derived for each sequence. This feature vector captured both the distribution and spatial arrangement of k-mers, preserving essential information about the sequence composition. The FCGR-based approach is particularly advantageous for analyzing DNA sequences as it integrates frequency and spatial information in a single representation. The ability to visualize and quantify k-mer distributions provides insights into sequence structures and patterns that might not be evident from raw sequence data. Furthermore, dividing the grid into sub-regions ensures that both local and global sequence characteristics are captured. Figure 5 is an example of the FCGR for different kmers.

2.5 Supplementary Features

A simple method for studying organisms and their relationships involves analyzing their unique genomic signatures [32]. These signatures can include basic attributes such as the GC content, GC skew, etc. In addition to the features derived from the encoding methods described earlier, this study incorporates GC content and GC skew, two biologically significant metrics that provide valuable insights into the genomic composition. These features complement the encoding methods described earlier, adding compositional context to the sequence data.

GC content represents the proportion of guanine (G) and cytosine (C) nucleotides in a given DNA sequence [33], defined as:

$$\text{GC Content} = \frac{\text{Number of G} + \text{Number of C}}{\text{Total Nucleotides}} \times 100, \quad (1)$$

whereas GC skew quantifies the asymmetry between guanine and cytosine nucleotides [33], calculated as:

$$\text{GC Skew} = \frac{\text{Number of G} - \text{Number of C}}{\text{Number of G} + \text{Number of C}}. \quad (2)$$

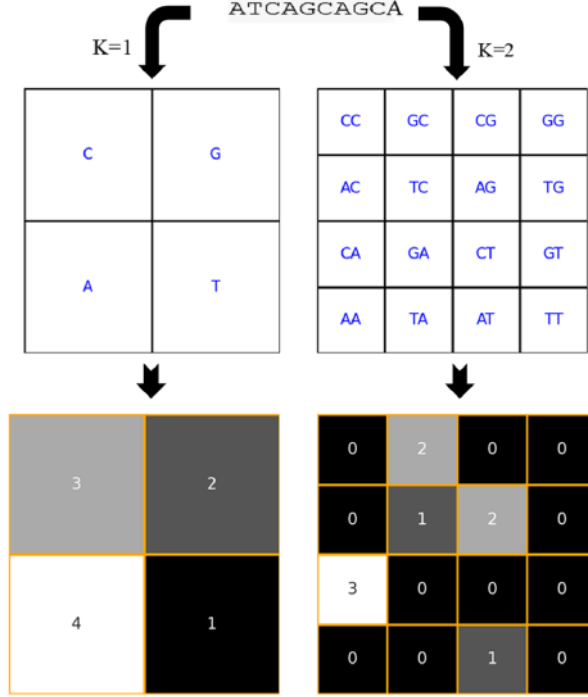


Fig. 5: Frequency Chaos game representation for varying kmers. On the left is the first-order FCGR ($k = 1$) and on the right is the second-order FCGR ($k = 2$).

In this study, GC content and GC skew were calculated for each sequence to investigate their role in differentiating viral genotypes. The GC content was primarily used as an indicator of sequence complexity and evolutionary pressure, while GC skew served as a proxy for replication-related asymmetry. These features were further integrated into downstream analyses, to evaluate their predictive power in distinguishing viral genotypes. The distributions of GC content across the genotypes, as illustrated in the provided density plots in (figure 6), reveal a diverse range of patterns and characteristics. Several genotypes, such as Genotype 1a and Genotype 5, exhibit tightly clustered distributions with sharp, narrow peaks. This indicates a high degree of consistency in GC content within these genotypes, likely reflecting stable and uniform genomic regions. For instance, Genotype 1a has a sharp peak centered around 60%, suggesting that most sequences within this genotype maintain a similar GC content. In contrast, broader distributions, such as those observed in Genotype 2c and Genotype 3b, indicate greater variability in GC content. The bimodal structure seen in Genotype 3b suggests the presence of distinct subpopulations or genomic regions with significantly different GC content values, possibly reflecting functional diversity or evolutionary adaptations within this genotype.

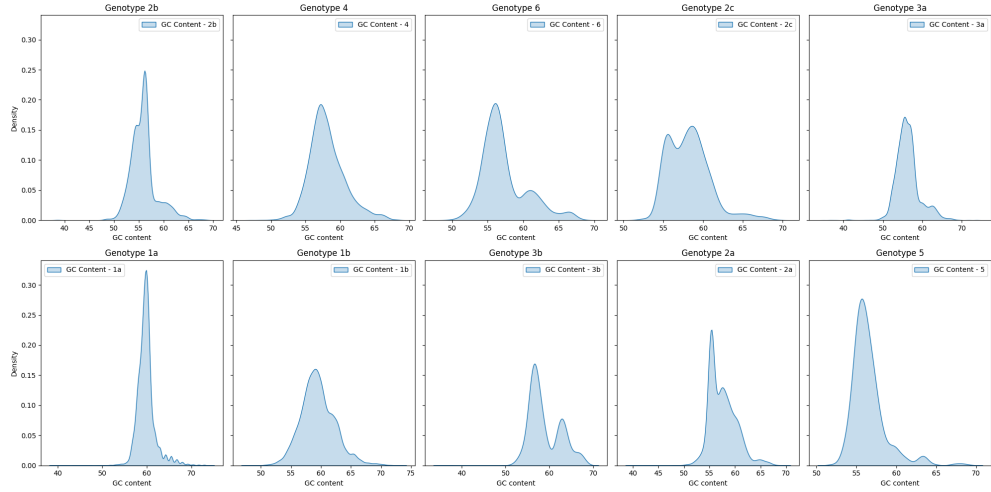


Fig. 6: GC Content of the different genotypes.

Some genotypes, such as Genotype 2a and Genotype 2b, have peaks slightly shifted from the typical range of 50–60%. For example, Genotype 2a shows a broader distribution with a secondary peak around 55%, suggesting heterogeneity in GC content. This could indicate varying functional across different regions of the genome. Similarly, Genotype 6 exhibits a broader range of GC content, peaking near 55% but with a significant tail extending towards higher values, implying a wider spectrum of sequence compositions. Interestingly, certain genotypes like Genotype 4 exhibit smoother distributions compared to others, possibly reflecting a more diverse sequence composition.

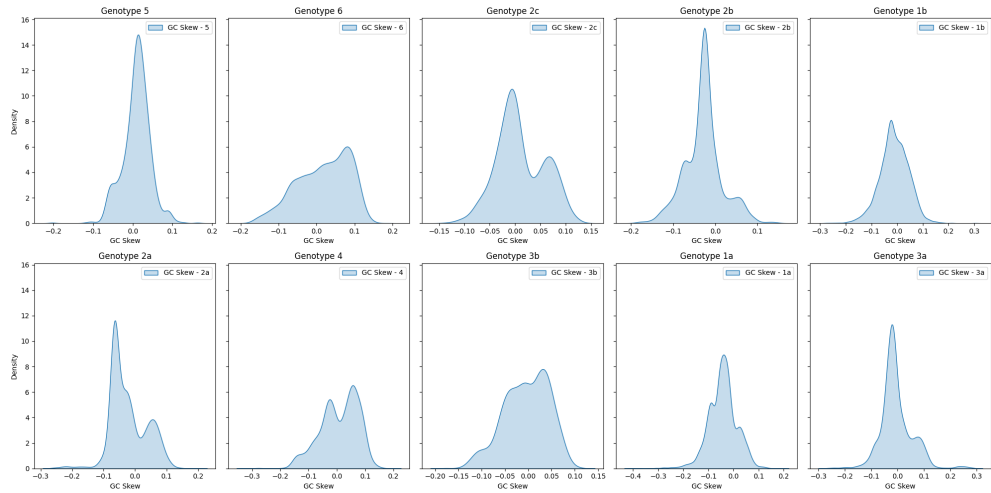


Fig. 7: GC Skew of the different genotypes.

It’s clear in Figure 7 the GC skew distributions across the genotypes reveal diverse patterns, offering insight into the underlying genomic characteristics of each genotype. Genotypes like Genotype 5 exhibit narrow and consistent distributions, indicating that GC skew values are relatively stable and clustered. In contrast, genotypes such as Genotype 6 display broader distributions, reflecting significant variability in GC skew, which may indicate more diverse genomic regions or a higher degree of evolutionary flexibility. Most genotypes follow a unimodal distribution pattern, where a single peak dominates, but some, like Genotype 4, show evidence of bimodality. This bimodal nature could signify subpopulations within the genotype or differences in the functional roles of various sequence regions contributing to the observed GC skew variability. Furthermore, the position of the peaks in the distributions provides additional insights. For example, genotypes such as Genotype 5 and Genotype 3a have peaks centered near zero, indicating a balanced representation of G and C nucleotides across their sequences. On the other hand, genotypes like Genotype 6 exhibit peaks shifted toward positive skew, suggesting an over representation of guanine relative to cytosine, while Genotype 2a displays peaks shifted toward negative skew, pointing to cytosine dominance. These differences in GC skew are indicative of varying biological processes, such as biases in replication or transcription, or specific genomic constraints affecting the nucleotide composition.

3 Results and Discussion

This section presents the results of integrating GC skew and GC content with various encoding methods and evaluates the effect of oversampling using SMOTE. The findings are analyzed to determine the contributions of these factors to the performance of machine learning models for sequence classification.

3.1 Experimental Setup

In our implementation, we utilized a colab CPU with a RAM size of 12 GB with no GPU was needed in our study. The dataset was initially divided into training and test sets to establish distinct subsets for model development and evaluation. To ensure a robust assessment of model performance, we employed a stratified five-fold cross-validation approach on the training portion. This methodology involved partitioning the training data into five equally sized folds while preserving the original class distribution within each subset. During the validation process, each fold served as the validation set in turn while the remaining four folds were used for model training, with this process repeated five times to ensure every data point contributed to both training and validation. The performance metrics obtained from each iteration were subsequently averaged to produce a comprehensive evaluation of model accuracy. This cross-validation strategy effectively minimizes bias that could arise from any single random data split while providing a reliable estimate of the model’s generalization capability. The preserved test set was then used exclusively for final model evaluation after all development and tuning procedures were completed. We employed the following formula for calculating the accuracy metric:

$$\text{ACC} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

Where, *True Positive (TP)* refers to the number of the coding sequences of the interested HCV genotype that were correctly predicted as the interested genotype by the model. *False Positive (FP)* refers to the number of coding sequences of non-interested HCV genotypes that were incorrectly predicted as the interested genotype by the model. *True Negative (TN)* refers to the number of the coding sequences of non-interested HCV genotypes correctly predicted as non-interested genotypes by the model. *False Negative (FN)* refers to the number of the coding sequences of the interested HCV genotype incorrectly predicted as non-interested genotypes by the model. where it evaluates the overall correctness of the predictions made by the model.

3.2 Model Selection

In this study, we selected a diverse set of machine learning models: KNN, RF, XGBoost, and MLP; to evaluate their effectiveness in HCV genotype prediction based on various genomic encoding schemes. These models were chosen to represent a balance between classical, ensemble-based, and neural network-based approaches, each offering complementary strengths in processing high-dimensional and imbalanced genomic data.

KNN was included as a simple yet effective baseline model due to its non-parametric nature and intuitive reliance on instance-based learning. While KNN often performs well in lower-dimensional spaces and can capture local data structures, it is sensitive to the choice of distance metrics and suffers from performance degradation in high-dimensional genomic datasets unless dimensionality reduction or effective feature extraction is applied. On the other hand, Random Forest, a robust ensemble method, has consistently demonstrated high accuracy in genomics tasks due to its ability to handle noisy features and capture non-linear relationships without extensive parameter tuning. Its built-in feature importance measures also aid in interpreting model decisions, an advantage in bioinformatics settings where explainability is critical.

XGBoost, a gradient boosting framework, was selected for its superior predictive power and regularization mechanisms, which help prevent overfitting—particularly important when working with complex and potentially redundant genomic features. Prior studies have shown XGBoost to be highly effective in biological classification problems, especially when combined with advanced encoding schemes such as k-mer encoding or FCGR-derived features. Lastly, MLPs offer a flexible deep learning architecture capable of learning hierarchical representations from raw genomic encodings. However, they are more sensitive to class imbalance and hyperparameter choices, and benefit greatly from oversampling techniques like SMOTE, as observed in our experiments. By evaluating this diverse model suite, we aim to provide a comprehensive understanding of the interplay between model complexity, feature representation, and class distribution in the context of HCV genotype prediction.

3.3 Analysis of FCGR and k-mer Encoding

To determine the most effective k configuration for both FCGR and k-mer encodings, we conducted two dedicated experiments. These experiments evaluated the performance of various machine learning models across multiple values of k ($k = 3, 4, 5$), under both imbalanced (original) and balanced (SMOTE-applied) settings.

Table 3: Cross validation accuracy (Mean $\pm$ Std) of FCGR encoding across various models.

Model	No SMOTE			With SMOTE		
	3	4	5	3	4	5
RF	0.9767 $\pm$ 0.0019	0.9837 $\pm$ 0.0020	0.9867 $\pm$ 0.0016	0.9807 $\pm$ 0.0017	0.9867 $\pm$ 0.0015	0.9891 $\pm$ 0.0012
XGBoost	0.9793 $\pm$ 0.0015	0.9881 $\pm$ 0.0017	0.9899 $\pm$ 0.0008	0.9802 $\pm$ 0.0016	0.9878 $\pm$ 0.0013	0.9901$\pm$0.0011
KNN	0.9786 $\pm$ 0.0018	0.9861 $\pm$ 0.0022	0.9862 $\pm$ 0.0013	0.9809 $\pm$ 0.0017	0.9873 $\pm$ 0.0020	0.9871 $\pm$ 0.0014
MLP	0.9408 $\pm$ 0.0073	0.9620 $\pm$ 0.0143	0.9784 $\pm$ 0.0084	0.9440 $\pm$ 0.0091	0.9812 $\pm$ 0.0049	0.9882 $\pm$ 0.0015

Table 4: Testset accuracy of FCGR encoding

Model	No SMOTE			With SMOTE		
	3	4	5	3	4	5
RF	0.9802	0.9850	0.9885	0.9836	0.9877	0.9902
XGBoost	0.9811	0.9893	0.9911	0.9820	0.9895	0.9908
KNN	0.9811	0.9865	0.9877	0.9814	0.9865	0.9881
MLP	0.9301	0.9802	0.9856	0.9340	0.9860	0.9895

Tables 3 and 4 summarize the cross-validation and hold-out test set results, respectively, for the FCGR-based representations. Across all configurations, the models demonstrated high predictive performance, with accuracy generally improving as the k size increased. Among the evaluated models, XGBoost consistently achieved the highest mean cross-validation accuracy, reaching a peak of 0.9901 ± 0.0011 at $k = 5$ with SMOTE. Random Forest also performed robustly across all k values, with accuracies closely trailing those of XGBoost. The application of SMOTE positively impacted performance, particularly in the case of MLP, which showed a substantial increase from 0.9784 ± 0.0084 (no SMOTE, $k = 5$) to 0.9882 ± 0.0015 (with SMOTE, $k = 5$), highlighting the importance of addressing class imbalance for neural network models. KNN exhibited relatively stable results across k values, with minor improvements under SMOTE.

Evaluation on the hold-out test set further validated the generalizability of the FCGR-based models. XGBoost once again achieved the highest test accuracy, reaching 0.9911 at $k = 5$ without SMOTE, with comparable performance when SMOTE

was applied (0.9908). These results confirm that the model is not overfitting and can generalize effectively to unseen data. Similarly, MLP showed notable improvements on the test set, with accuracy rising from 0.9856 to 0.9895 at $k = 5$ after SMOTE. While RF and KNN also performed well, in some cases they achieved slightly higher accuracy without SMOTE, indicating that the impact of oversampling can vary depending on the algorithm and underlying data distribution.

Table 5: Cross validation accuracy (Mean $\pm$ Std) of K-mer encoding.

Model	No SMOTE			With SMOTE		
	3	4	5	3	4	5
RF	0.9732 $\pm$ 0.0007	0.9816 $\pm$ 0.0020	0.9858 $\pm$ 0.0016	0.9807 $\pm$ 0.0016	0.9868 $\pm$ 0.0019	0.9899$\pm$0.0014
XGBoost	0.9793 $\pm$ 0.0015	0.9793 $\pm$ 0.0015	0.9793 $\pm$ 0.0015	0.9802 $\pm$ 0.0016	0.9802 $\pm$ 0.0016	0.9802 $\pm$ 0.0016
KNN	0.9238 $\pm$ 0.0046	0.9809 $\pm$ 0.0017	0.9786 $\pm$ 0.0018	0.9809 $\pm$ 0.0017	0.9786 $\pm$ 0.0018	0.9809 $\pm$ 0.0017
MLP	0.9231 $\pm$ 0.0133	0.9556 $\pm$ 0.0154	0.9860 $\pm$ 0.0052	0.9410 $\pm$ 0.0075	0.9698 $\pm$ 0.0102	0.9874 $\pm$ 0.0013

Table 6: Testset accuracy of K-mer encoding.

Model	No SMOTE			With SMOTE		
	3	4	5	3	4	5
RF	0.9761	0.9837	0.9874	0.9816	0.9878	0.9904
XGBoost	0.9811	0.9811	0.9811	0.9820	0.9820	0.9820
KNN	0.9309	0.9814	0.9811	0.9814	0.9811	0.9814
MLP	0.9327	0.9840	0.9893	0.9475	0.9681	0.9882

Tables 5 and 6 present the results for models trained using k-mer encoding. The trends observed here are consistent with those of the FCGR-based experiments. Random Forest achieved the highest cross-validation accuracy, peaking at 0.9899 $\pm$ 0.0014 for $k = 5$ with SMOTE. XGBoost maintained stable performance across all k-mer sizes and SMOTE conditions, with accuracies ranging between 0.9793 and 0.9802. The use of SMOTE yielded moderate improvements in most cases, particularly for MLP, which showed a considerable increase in accuracy from 0.9860 $\pm$ 0.0052 to 0.9874 $\pm$ 0.0013 at $k = 5$. KNN demonstrated a significant accuracy jump between $k = 3$ and $k = 4$, after which performance plateaued, indicating that higher-order k-mers contribute more discriminative power.

Hold-out test set performance confirmed the robustness of the models trained on k-mer features. RF once again achieved the highest accuracy, reaching 0.9904 at $k = 5$ with SMOTE, while XGBoost sustained its performance across all conditions, with test accuracy consistently around 0.9820. MLP also performed strongly, demonstrating high accuracy both with and without SMOTE, while KNN showed minimal

performance change with oversampling, reflecting its resilience to moderate class imbalance.

In summary, the results from both FCGR and k-mer encoding experiments consistently prove that larger k-mer sizes ($k = 5$) enhance model performance by capturing richer contextual patterns in the genomic sequences. Ensemble learning models, particularly XGBoost and Random Forest, emerged as the most robust and accurate classifiers across both encoding strategies. SMOTE further improved performance, especially for models sensitive to data imbalance such as MLP. The close alignment between cross-validation and hold-out test set accuracies supports the models' ability to generalize well, with no indication of overfitting.

3.4 Encoding and Oversampling

Tables 7 and 8 provide a comprehensive comparison of four encoding strategies: FCGR with $k = 5$, one-hot, label encoding, and k-mer with $k = 5$, across four machine learning models: RF, XGBoost, KNN, and MLP. The analysis is conducted under two scenarios: with and without SMOTE for addressing class imbalance. As shown in Table 7, the FCGR and k-mer encodings consistently outperform the simpler one-hot and label encodings across all models and imbalance settings. FCGR achieved the highest classification accuracy at 0.9901 ± 0.001 using SMOTE. On the other hand, without SMOTE, XGBoost achieved the highest accuracy of 0.9899 ± 0.0008 using FCGR, while Random Forest reached 0.9858 ± 0.0016 using k-mer encoding. The application of SMOTE improved the overall model performance across most settings, especially for MLP and KNN. For example, MLP improved from 0.9784 ± 0.0084 (FCGR, no SMOTE) to 0.9882 ± 0.0015 with SMOTE, and from 0.9860 ± 0.0052 (k-mer, no SMOTE) to 0.9874 ± 0.0013 with SMOTE. These results prove that more complex encodings such as FCGR and k-mers can better capture sequence-level features critical for accurate HCV genotype classification.

Table 8 confirms the generalization capability of the observed trends. On the unseen test set, both FCGR and k-mer encodings continued to yield the highest accuracy, with Random Forest and XGBoost leading the performance. Notably, XGBoost achieved the best accuracy of 0.9911 on FCGR without SMOTE, while RF reached a peak of 0.9904 using k-mer encoding with SMOTE. These high scores suggest that both FCGR and k-mer encodings not only enhance training accuracy but also generalize well on external test data. One-hot and label encodings showed substantially lower performance, particularly with MLP, where label encoding produced only 0.9144 accuracy without SMOTE, compared to 0.9882 using k-mer encoding. Across nearly all encoding methods, the integration of SMOTE improved classification accuracy, especially for MLP, which is generally more sensitive to class imbalance. However, for robust ensemble methods like RF and XGBoost, the effect of SMOTE was more subtle but still favorable in most cases. Interestingly, in a few configurations such as XGBoost on FCGR, SMOTE slightly decreased test accuracy (0.9911 to 0.9908), suggesting potential over-saturation or redundancy introduced by synthetic samples.

Overall, FCGR and k-mer encodings clearly show superior power in modeling genomic sequences for HCV genotyping. Among models, ensemble methods such as XGBoost and Random Forest consistently deliver the highest performance, while MLP shows noticeable improvement when SMOTE is applied. One-hot and label encodings, although straightforward to implement, may not capture complex sequence patterns effectively, limiting their predictive capabilities in high-dimensional biological datasets.

These findings support the conclusion that appropriate sequence encoding, coupled with imbalance handling, plays a crucial role in building robust models for HCV genotyping. FCGR and k-mer encodings, especially with $k = 5$, are validated as effective feature representations when paired with ensemble classifiers and oversampling techniques.

Table 7: Cross-validation comparison across models with and without SMOTE. The overall best result is **boldfaced** (LE: Label Encoding).

SMOTE	Model	FCGR	One-hot	LE	k-mer
No	RF	.9867 $\pm$.001	.9500 $\pm$.002	.9552 $\pm$.001	.9858 $\pm$.001
	XGB	.9899 $\pm$.000	.9579 $\pm$.001	.9575 $\pm$.001	.9793 $\pm$.001
	KNN	.9862 $\pm$.001	.9248 $\pm$.001	.9238 $\pm$.001	.9786 $\pm$.001
	MLP	.9784 $\pm$.008	.9357 $\pm$.003	.9003 $\pm$.005	.9860 $\pm$.0052
Yes	RF	.9891 $\pm$.001	.9570 $\pm$.002	.9561 $\pm$.000	.9899 $\pm$.001
	XGB	.9901$\pm$.001	.9576 $\pm$.001	.9576 $\pm$.001	.9802 $\pm$.001
	KNN	.9871 $\pm$.001	.9289 $\pm$.001	.9277 $\pm$.000	.9809 $\pm$.001
	MLP	.9882 $\pm$.0015	.935 $\pm$.003	.9125 $\pm$.004	.9874 $\pm$.001

Table 8: Performance comparison on the testset across models With and without SMOTE. The overall best result is **boldfaced**.

Model	No SMOTE				With SMOTE			
	FCGR	One-hot	Label	k-mer	FCGR	One-hot	Label	k-mer
RF	0.9885	0.9568	0.9578	0.9874	0.9902	0.9564	0.9603	0.9904
XGB	0.9911	0.9583	0.9606	0.9811	0.9908	0.9605	0.9605	0.9820
KNN	0.9877	0.9274	0.9302	0.9811	0.9881	0.9281	0.9349	0.9814
MLP	0.9895	0.9355	0.9144	0.9882	0.9856	0.9418	0.9153	0.9893

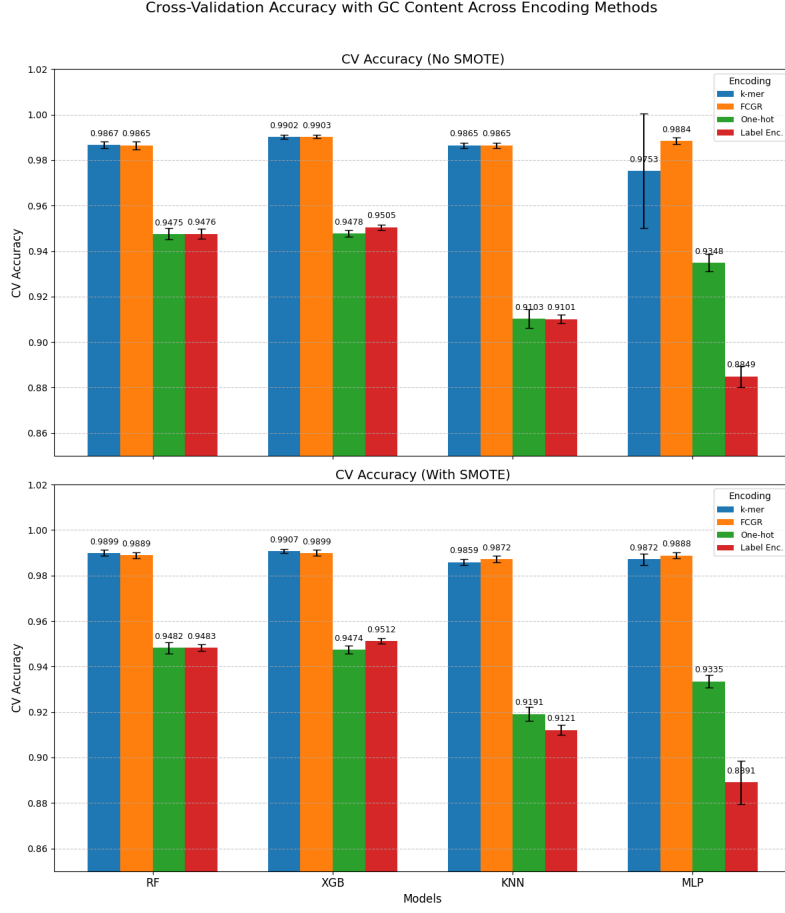


Fig. 8: Cross-validation accuracy comparison across encoding methods (k-mer, FCGR, One-hot, and Label Encoding) after integrating the GC Content feature. The top panel presents accuracy results without SMOTE, and the bottom panel displays results obtained after applying SMOTE. Accuracy scores are indicated on top of each bar, with error bars highlighting variability across cross-validation folds.

3.5 Supplementary Features

Across both GC Content and GC Skew features, the encoding methods significantly influence classification performance. Figures 8 and 9 show the result of the cross validation after integrating the GC content and GC skewness with the input features. It is clear that numerical encodings (k-mer and FCGR) consistently outperform simpler categorical encodings (*One-hot* and Label Encoding). Specifically, k-mer and

Cross-Validation Accuracy with GC Skew Across Encoding Methods

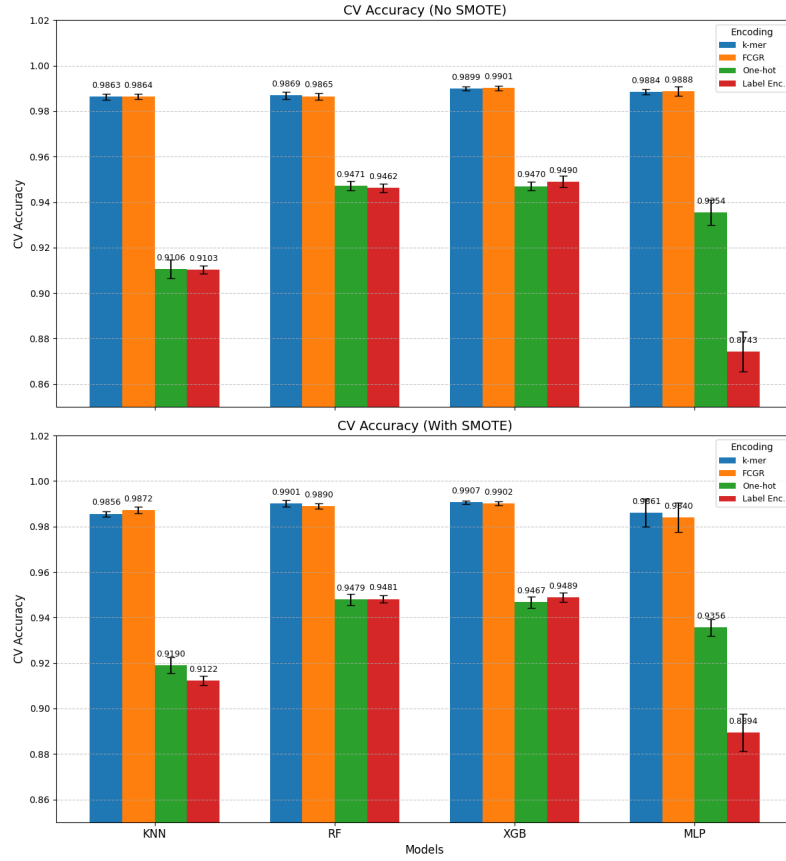


Fig. 9: Cross-validation accuracy comparison across encoding methods (k-mer, FCGR, One-hot, and Label Encoding) after integrating the GC Skew feature. The upper panel shows results without SMOTE, while the lower panel illustrates performance with SMOTE applied. The bar heights indicate the accuracy scores, with error bars representing the variability in performance.

FCGR encodings demonstrated cross-validation accuracies around (98%-99%), compared to considerably lower results (91%-95%) observed with One-hot and Label Encoding.

Comparing GC Content and GC Skew revealed slight differences in model performance. Generally, GC Content yielded marginally higher or comparable accuracies compared to GC Skew, particularly evident with XGBoost, achieving CV accuracies as high as 99.07% and test accuracies up to 99.25% when SMOTE was applied. In contrast, GC Skew provided consistently strong, yet slightly lower performance, especially within neural network models such as MLP. Thus, GC Content appears

to offer a modest predictive advantage, especially when paired with advanced modeling approaches like XGBoost.

The application of SMOTE generally improved model stability and accuracy, particularly benefiting neural network architectures. For instance, using GC Content, the MLP CV accuracy notably improved from $97.53\% \pm 2.53\%$ without SMOTE to $98.72\% \pm 0.25\%$ with SMOTE, accompanied by significant gains in test accuracy from 96.44% to 98.93%. Similar trends emerged for GC Skew, notably increasing the test accuracy of XGB from 99.22% without SMOTE to 99.29% with SMOTE, indicating SMOTE’s effectiveness in mitigating class imbalance and enhancing predictive consistency.

Evaluating the relative performance of different models, we found that XGBoost consistently achieved superior predictive accuracy and stability, maintaining CV and test accuracies above 99%. Random Forest and KNN performed comparably, though Random Forest often slightly outperformed KNN. The MLP showed notable sensitivity to encoding method and class imbalance conditions but displayed substantial performance gains when SMOTE was utilized.

An examination of test accuracies further highlighted the models’ generalization capabilities. XGBoost, particularly with k-mer encoding, reached the highest test accuracies of 99.25% for GC Content and 99.29% for GC Skew under SMOTE conditions. Conversely, models utilizing One-hot and Label Encoding methods consistently demonstrated reduced test accuracies, confirming the importance of careful encoding selection.

Analysis of variability through the standard deviation of CV accuracy highlighted model stability. The application of SMOTE consistently lowered variability across models, notably in MLP for GC Content, where variability dramatically reduced from $\pm 0.55\%$ to $\pm 0.25\%$. Ensemble methods, including XGBoost and Random Forest, exhibited inherently low variability (typically below 0.2%), highlighting their robustness in genomic classification tasks.

In summary, our findings provide several practical recommendations for genomic predictive modeling: prioritize numeric encoding methods such as k-mer and FCGR, employ SMOTE to address class imbalance effectively, and utilize robust ensemble methods like XGBoost for optimal performance. These recommendations collectively highlight crucial methodological considerations to enhance the reliability and accuracy of predictive genomic models.

Table 9: Test accuracy across models and encoding methods using GC Content, split by SMOTE condition (LE: Label Encoding).

Model	No SMOTE				Model	With SMOTE			
	k-mer	FCGR	One-hot	LE		k-mer	FCGR	One-hot	LE
KNN	0.9882	0.9879	0.9193	0.9198	KNN	0.9872	0.9880	0.9266	0.9246
RF	0.9893	0.9889	0.9526	0.9555	RF	0.9908	0.9904	0.9547	0.9577
XGB	0.9916	0.9910	0.9527	0.9601	XGB	0.9925	0.9917	0.9527	0.9595
MLP	0.9644	0.9869	0.9423	0.8937	MLP	0.9893	0.9871	0.9420	0.9058

Table 10: Test accuracy across models and encoding methods using GC Skew, split by SMOTE condition (LE: Label Encoding).

No SMOTE					With SMOTE				
Model	k-mer	FCGR	One-hot	LE	Model	k-mer	FCGR	One-hot	LE
KNN	0.9879	0.9875	0.9193	0.9196	KNN	0.9872	0.9882	0.9271	0.9241
RF	0.9880	0.9882	0.9538	0.9551	RF	0.9909	0.9899	0.9528	0.9564
XGB	0.9922	0.9915	0.9541	0.9584	XGB	0.9929	0.9918	0.9550	0.9595
MLP	0.9814	0.9877	0.9447	0.8787	MLP	0.9885	0.9885	0.9421	0.8910

3.6 Comparative Analysis of Model Performance and Resource Efficiency

Table 11 summarizes the per-class performance metrics of five different models applied to the task of genotype classification. All models except CNN were trained using k-mer encoding, the best encoding method concluded from the previous section, in addition to the CNN was trained to provide a deep learning benchmark. Classical machine learning models, especially Random Forest and XGBoost, exhibited consistently high classification performance across all genotypes. XGBoost yielded the highest F1-scores overall, with particularly strong results for classes 2a (F1-score = 0.973), 5 (0.980), and 6 (0.971). Random Forest followed closely with comparable F1-scores across these same classes. KNN also demonstrated competitive performance, especially in recall and specificity metrics, showing strong generalization for most genotypes while being computationally simpler than ensemble models. Notably, the KNN model achieved an F1-score of 0.986 for class 3a and 0.981 for class 6, closely matching the best-performing models.

On the other hand, The MLP, in contrast, showed markedly lower performance. Its F1-scores were considerably reduced across all classes, with particularly poor values for classes 1b (0.034), 2a (0.101), and 5 (0.083). This highlights the model’s difficulty in capturing complex sequence-level patterns using only dense layers without convolutional or recurrent components, even when trained with k-mer features. Interestingly, the 1D CNN model, achieved strong F1-scores for several classes, notably class 3b (0.96), class 5 (0.97), and class 6 (0.96). This implies that the convolutional architecture was effective in learning local sequential patterns even from a basic representation. However, for more complex or overlapping classes like 2a or 2b, its performance (F1 = 0.95) was slightly lower to that of XGBoost.

Notably, genotypes such as 2c, 3b, and 5 had substantially fewer samples in the original dataset (717, 1,157, and 1,036 respectively), potentially posing a challenge for accurate classification due to limited training data.

It is clear that per-genotype performance across different models using k-mer encoding. Models like Random Forest and XGBoost maintained high accuracy and recall even for the underrepresented genotypes. For example, XGBoost

Table 11: Per-class performance metrics for different models using K-mer encoding.

Model	Class	Accuracy	Precision	Recall	Specificity	F1-score
KNN	1a	0.994	0.979	0.991	0.995	0.985
	1b	0.992	0.980	0.980	0.995	0.980
	2a	0.998	0.978	0.976	0.999	0.977
	2b	0.998	0.972	0.984	0.999	0.978
	2c	0.999	0.961	0.972	0.999	0.967
	3a	0.995	0.982	0.990	0.996	0.986
	3b	0.997	0.955	0.958	0.998	0.957
	4	0.995	0.989	0.973	0.998	0.981
	5	0.998	0.977	0.969	0.999	0.973
	6	0.996	0.991	0.971	0.999	0.981
Random Forest	1a	0.995	0.987	0.990	0.997	0.988
	1b	0.989	0.957	0.988	0.990	0.972
	2a	0.997	0.985	0.959	0.999	0.972
	2b	0.998	0.987	0.961	0.999	0.974
	2c	0.998	0.994	0.911	1.000	0.950
	3a	0.995	0.980	0.991	0.995	0.986
	3b	0.997	0.982	0.920	0.999	0.950
	4	0.992	0.973	0.964	0.996	0.968
	5	0.999	0.988	0.961	1.000	0.975
	6	0.994	0.977	0.970	0.997	0.974
XGBoost	1a	0.994	0.982	0.990	0.996	0.986
	1b	0.991	0.970	0.983	0.993	0.977
	2a	0.997	0.988	0.959	0.999	0.973
	2b	0.997	0.971	0.963	0.999	0.967
	2c	0.998	0.994	0.927	1.000	0.960
	3a	0.995	0.983	0.989	0.996	0.986
	3b	0.997	0.971	0.924	0.999	0.947
	4	0.992	0.967	0.968	0.995	0.967
	5	0.999	1.000	0.961	1.000	0.980
	6	0.993	0.969	0.972	0.996	0.971
MLP	1a	0.807	0.113	0.108	0.894	0.111
	1b	0.827	0.035	0.033	0.908	0.034
	2a	0.822	0.097	0.105	0.897	0.101
	2b	0.830	0.115	0.133	0.899	0.124
	2c	0.812	0.144	0.164	0.887	0.153
	3a	0.808	0.089	0.100	0.887	0.094
	3b	0.834	0.087	0.089	0.908	0.088
	4	0.827	0.114	0.130	0.898	0.122
	5	0.822	0.105	0.068	0.923	0.083
	6	0.829	0.175	0.157	0.910	0.166
CNN	1a	0.99	0.98	0.99	0.99	0.98
	1b	0.99	0.98	0.98	0.99	0.98
	2a	0.98	0.93	0.98	0.99	0.95
	2b	0.98	0.94	0.96	0.99	0.95
	2c	0.97	0.92	0.94	0.99	0.93
	3a	1.00	0.95	0.96	0.99	0.96
	3b	1.00	0.96	0.94	0.99	0.96
	4	0.94	0.92	0.94	0.99	0.96
	5	0.91	0.96	0.98	0.99	0.97
	6	0.92	0.96	0.94	0.99	0.96

achieved 0.998 accuracy and 0.927 recall for genotype 2c, and 0.997 accuracy with 0.924 recall for 3b, indicating effective generalization despite the initial class imbalance. Similarly, genotype 5 reached 0.999 accuracy with 0.961 recall under XGBoost, demonstrating the model’s ability to distinguish low-frequency patterns when supplemented with SMOTE and informative features.

From the resource usage point of view, Table 12 reports training and inference time alongside memory usage for each model. The KNN model, while fast to train (1.29 s), exhibited the highest inference memory demand (7190 MiB) and inference time (60 s), owing to its lazy evaluation mechanism that requires storing the entire dataset.

Random Forest and XGBoost required significantly more training time (19.17 s and 91.22 s, respectively), but had fast and memory-efficient inference, making them well-suited for deployment scenarios where inference speed is critical.

MLP and CNN showed the lowest memory usage overall. The CNN required slightly more training time (51 s) than the MLP (1.5 s) but still much less than the tree-based models. The CNN’s inference was very fast (1.06 s), and the memory usage remained minimal (16.3 MiB), highlighting its suitability for lightweight deployment.

In summary, the results indicate that XGBoost and Random Forest with k-mer encoding offer the best trade-off between accuracy and generalization for genotype classification. While deep learning with CNNs using label encoding demonstrated promising performance, especially with limited resources, it still underperformed slightly compared to the k-mer-based ensemble models. The MLP model proved inadequate in both performance and generalization. Performance and memory metrics using the other encoding methods can be found in the Appendix.

Table 12: Training and inference time (in seconds) and memory usage (in MiB) for the different models using k-mer encoding.

Model	Train Time	Infer. Time	Train Memory	Infer. Memory
KNN	1.2921	60.0718	5267.13	7190.50
Random Forest	19.1658	0.8362	5383.74	5289.88
XGBoost	91.2241	0.7153	5409.83	5332.55
MLP	1.5006	0.7007	52.42	55.18
CNN	51.0000	1.0600	186.40	16.30

3.7 Statistical Significance

To assess the impact of oversampling and supplementary genomic features on model performance, we conducted paired t-tests comparing cross-validation accuracies under two conditions: (1) with SMOTE and GC content, and (2) with SMOTE and GC skew, against a baseline configuration that excluded both. The analysis was performed independently for each encoding scheme to determine whether the observed improvements were statistically significant.

In the first configuration, where SMOTE was applied in combination with GC content, the results indicate statistically significant improvements for both FCGR ($t = 3.192$, $p = 0.0018$) and k-mer encoding ($t = 14.859$, $p \approx 6.95 \times 10^{-29}$). These findings imply that the integration of GC content substantially enhances model performance when paired with informative encoding schemes capable of capturing compositional and spatial dependencies. In contrast, One-hot encoding ($t = -1.265$, $p = 0.208$) and Label encoding ($t = -4.160$, $p \approx 6.04 \times 10^{-5}$) either failed to benefit or showed negative impacts. The latter may be attributed to the limited representational capacity of these encodings, which could not effectively leverage the added biological signal.

In the second configuration, evaluating the effect of GC skew with SMOTE, similarly strong gains were observed for k-mer ($t = 12.623$, $p < 0.0001$) and FCGR ($t = 6.808$, $p < 0.0001$). These results underscore the robustness of these encoding strategies in capturing biological variations. However, for One-hot encoding ($t = -2.000$, $p = 0.048$), the performance difference was only marginally significant, while Label encoding remained statistically insignificant ($t = -1.721$, $p = 0.089$).

These results collectively highlight that while the addition of the supplementary features can significantly enhance classification performance, the extent of benefit is highly dependent on the underlying sequence encoding. Encoding strategies like k-mer and FCGR are better suited to utilize such information effectively, offering a promising direction in viral sequence classification tasks.

3.8 Feature Importance Analysis

To deepen our understanding of the decision processes behind the evaluated models, we examined the feature importance scores produced by both XGBoost and Random Forest when trained using k-mer frequencies combined with either GC content or GC skew. Figure 10 illustrates the top 20 features for each model-feature configuration, highlighting the most influential elements that guided the classification of HCV genotypes.

When analyzing the XGBoost model trained with k-mer features and GC skew, we observed that the model strongly favored individual k-mers. In particular, the k-mer **ttaag** emerged as the most influential feature, followed by others such as **gggtg**, **acata**, and **gtcct**. Interestingly, the GC skew metric itself did not appear among the top-ranked features, implying that XGBoost prioritized highly discriminative k-mer motifs over broader compositional trends. This behavior is consistent with

XGBoost’s tendency to exploit localized feature patterns with strong individual predictive value.

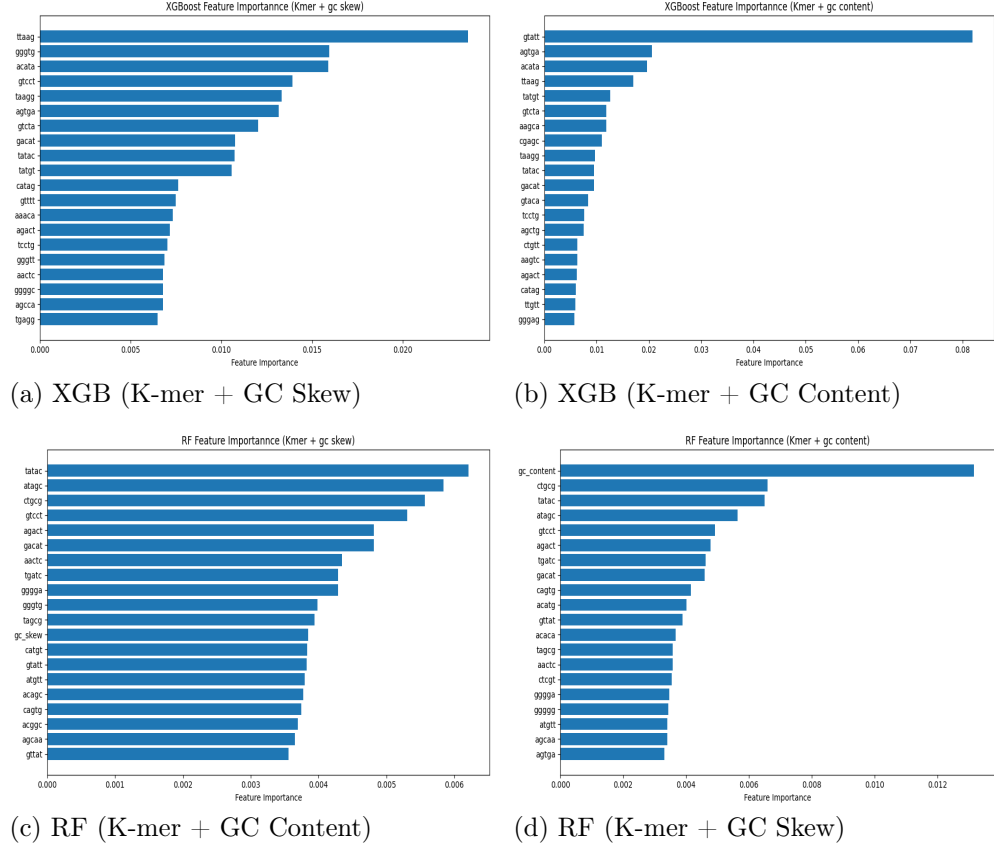


Fig. 10: Comparison of top 20 most important features using Random Forest and XGBoost models across two configurations: K-mer with GC content and K-mer with GC skew.

In contrast, the XGBoost model trained with GC content revealed a slightly different picture. Here, the feature **gtatt** showed a disproportionately high importance, dominating the top of the ranking. Other relevant k-mers included **agtga**, **acata**, and **ttaag**, yet none approached the magnitude of influence held by **gtatt**. Again, the GC content feature itself was not prominent among the most impactful features. These findings indicate that XGBoost focuses on a sparse set of highly influential sequence motifs and tends to disregard global composition metrics, even when they are provided.

On the other hand, the Random Forest model demonstrated a more balanced approach to feature importance. When trained with k-mer frequencies and GC content, the model ranked GC content as the most important feature, followed by a mix of k-mers such as `ctgcg`, `tatac`, and `atagc`. This implies that Random Forest considers both global nucleotide composition and local motifs in its predictive process. The distribution of feature importance values was also more uniform compared to XGBoost, which implies that RF builds its predictions on a broader set of moderately informative features rather than relying on a few dominant signals. A similar trend was observed when Random Forest was trained with GC skew. Although GC skew ranked lower than it did in the GC content configuration, it still appeared among the top twenty features, trailing behind several k-mers such as `tatac`, `atagc`, and `gtcct`. The relative decrease in GC skew’s importance implies that while it contributes meaningfully to the classification task, its influence is secondary to specific sequence motifs. Nevertheless, Random Forest demonstrated a consistent capacity to integrate both local and global descriptors into its learning strategy.

Collectively, these observations highlight distinct modeling behaviors between XGBoost and Random Forest. XGBoost tends to focus narrowly on a few high-impact k-mers, largely neglecting global sequence features such as GC content and skew. Random Forest, in contrast, more evenly distributes its attention between k-mer patterns and compositional attributes, particularly in the case of GC content. These results indicate that the choice of model can significantly influence which aspects of the genomic representation are emphasized, and that hybrid modeling strategies may benefit from leveraging the complementary strengths of both approaches.

4 Conclusion

This study systematically evaluated the combined impact of encoding techniques, supplementary genomic features, and class-balancing on the genotyping of HCV sequences. Our experiments demonstrate that advanced representations such as k-mer encoding and FCGR substantially outperform simpler methods like One-hot and Label encoding, by more effectively capturing compositional and spatial patterns inherent in nucleotide sequences. Compared to previous studies that often rely on single encoding approaches, our work provides a comprehensive and comparative evaluation across diverse strategies.

Among the machine learning models explored, XGBoost and Random Forest consistently delivered superior accuracy, particularly when combined with k-mer and FCGR encodings. These tree-based models also maintained favorable computational efficiency, proposing practical advantages for real-world deployment. In contrast, while KNN achieved competitive accuracy, its inference latency and memory demands limit its scalability. MLP showed marked improvement only when enhanced by appropriate encoding and class balancing, whereas a lightweight 1D

CNN with Label encoding offered an efficient deep learning alternative, albeit slightly behind ensemble methods in accuracy. Importantly, integration of biologically relevant supplementary features, GC content and GC skew, was shown to further enhance model performance, especially in the case of FCGR and kmer. This implies that augmenting encodings with genomic descriptors enriches feature spaces and supports more accurate classification. Furthermore, the use of SMOTE was essential for correcting class imbalance, significantly boosting performance for underrepresented genotypes, an issue that has often been overlooked in previous HCV genotyping studies. Future work should extend this approach by incorporating interpretable AI techniques such as attention mechanisms or SHAP values to explain the underlying biological mechanisms that inform model predictions. Furthermore, applying this framework to other genomic classification tasks could broaden its impact. By emphasizing interpretability, scalability, and performance on imbalanced datasets, this study provides a significant step forward in the application of machine learning to viral genomics.

Data availability

The genome sequences used in this study are available in the Los-Alamos repository, <https://hcv.lanl.gov/content/sequence/HCV/ToolsOutline.html>, and the accession numbers of these sequences are available as supplementary material.

References

- [1] Françoise Roudot-Thoraval. “Epidemiology of hepatitis C virus infection”. In: *Clinics and research in hepatology and gastroenterology* 45.3 (2021), p. 101596.
- [2] Sarah Blach et al. “Global change in hepatitis C virus prevalence and cascade of care between 2015 and 2020: a modelling study”. In: *The Lancet Gastroenterology & Hepatology* 7.5 (2022), pp. 396–415.
- [3] Tania M Welzel et al. “Global epidemiology of HCV subtypes and resistance-associated substitutions evaluated by sequencing-based subtype analyses”. In: *Journal of hepatology* 67.2 (2017), pp. 224–236.
- [4] Tarik Asselah et al. “Direct-acting antivirals for the treatment of hepatitis C virus infection: optimizing current IFN-free treatment and future perspectives”. In: *Liver International* 36 (2016), pp. 47–57.
- [5] H James Hnatyszyn. “Chronic hepatitis C and genotyping: the clinical significance of determining HCV genotypes”. In: *Antiviral therapy* 10.1 (2005), pp. 1–11.
- [6] Jane P Messina et al. “Global distribution and prevalence of hepatitis C virus genotypes”. In: *Hepatology* 61.1 (2015), pp. 77–87.
- [7] Miguel Angel Martinez and Sandra Franco. “Therapy implications of hepatitis C virus genetic diversity”. In: *Viruses* 13.1 (2020), p. 41.
- [8] Hubert D-J Daniel et al. “Comparison of Three Different Hepatitis C Virus Genotyping Methods: 5’ NCR PCR-RFLP, Core Type-Specific PCR, and NS

- 5b Sequencing in a Tertiary Care Hospital in South India". In: *Journal of Clinical Laboratory Analysis* 31.3 (2017), e22045.
- [9] Frederick S Nolte et al. "Clinical evaluation of two methods for genotyping hepatitis C virus based on analysis of the 5' noncoding region". In: *Journal of clinical microbiology* 41.4 (2003), pp. 1558–1564.
 - [10] Tao Ai et al. "Correlation of chest CT and RT-PCR testing for coronavirus disease 2019 (COVID-19) in China: a report of 1014 cases". In: *Radiology* 296.2 (2020), E32–E40.
 - [11] Buddhisha Udugama et al. "Diagnosing COVID-19: the disease and tools for detection". In: *ACS nano* 14.4 (2020), pp. 3822–3835.
 - [12] Yong-Hak Sohn et al. "Performance evaluation of the Abbott RealTi me HCV Genotype II for hepatitis C virus genotyping". In: *Clinical chemistry and laboratory medicine* 48.4 (2010), pp. 469–474.
 - [13] Rashid Naseem et al. "Performance assessment of classification algorithms on early detection of liver syndrome". In: *Journal of Healthcare Engineering* 2020.1 (2020), p. 6680002.
 - [14] Reza Safdari et al. "Applying data mining techniques to classify patients with suspected hepatitis C virus infection". In: *Intelligent Medicine* 2.04 (2022), pp. 193–198.
 - [15] Umesh Kumar Lilhore et al. "Hybrid model for precise hepatitis-C classification using improved random forest and SVM method". In: *Scientific Reports* 13.1 (2023), p. 12473.
 - [16] Yongxian Fan, Xiqian Lu, and Guicong Sun. "IHCP: interpretable hepatitis C prediction system based on black-box machine learning models". In: *BMC bioinformatics* 24.1 (2023), p. 333.
 - [17] Chaozhi Li. "Predictors selection strategy based on stepwise random forests and logistic regression model". In: *International conference on statistics, data science, and computational intelligence (CSDSCI 2022)*. Vol. 12510. SPIE. 2023, pp. 251–256.
 - [18] Rok Blagus and Lara Lusa. "SMOTE for high-dimensional class-imbalanced data". In: *BMC bioinformatics* 14 (2013), pp. 1–16.
 - [19] Mete Yağanoğlu. "Hepatitis C virus data analysis and prediction using machine learning". In: *Data & Knowledge Engineering* 142 (2022), p. 102087.
 - [20] Heba Mamdouh Farghaly, Mahmoud Y Shams, and Tarek Abd El-Hafeez. "Hepatitis C Virus prediction based on machine learning framework: a real-world case study in Egypt". In: *Knowledge and Information Systems* 65.6 (2023), pp. 2595–2617.
 - [21] Davide Chicco and Giuseppe Jurman. "An ensemble learning approach for enhanced classification of patients with hepatitis and cirrhosis". In: *IEEE Access* 9 (2021), pp. 24485–24498.
 - [22] Azadeh Alizargar, Yang-Lang Chang, and Tan-Hsu Tan. "Performance comparison of machine learning approaches on hepatitis C prediction employing data mining techniques". In: *Bioengineering* 10.4 (2023), p. 481.
 - [23] Ping Qiu et al. "HCV genotyping using statistical classification approach". In: *Journal of biomedical science* 16 (2009), pp. 1–9.

- [24] Peilin Xie et al. “CAP-m7G: A capsule network-based framework for specific RNA N7-methylguanosine site identification using image encoding and reconstruction layers”. In: *Computational and Structural Biotechnology Journal* 27 (2025), pp. 804–812.
- [25] Lantian Yao et al. “CapsEnhancer: an effective computational framework for identifying enhancers based on chaos game representation and capsule network”. In: *Journal of Chemical Information and Modeling* 64.14 (2024), pp. 5725–5736.
- [26] Ahmed M Fahmy et al. “On leveraging self-supervised learning for accurate HCV genotyping”. In: *Scientific reports* 14.1 (2024), p. 15463.
- [27] Hania Ahmed et al. “ViroNia: LSTM based proteomics model for precise prediction of HCV”. In: *Computers in Biology and Medicine* 186 (2025), p. 109573. DOI: [10.1016/j.combiomed.2024.109573](https://doi.org/10.1016/j.combiomed.2024.109573). URL: <https://www.sciencedirect.com/science/article/pii/S0010482524016585>.
- [28] Carla Kuiken et al. “The hepatitis C sequence database in Los Alamos”. In: *Nucleic acids research* 36.suppl_1 (2007), pp. D512–D516.
- [29] Carla Kuiken et al. “The Los Alamos hepatitis C sequence database”. In: *Bioinformatics* 21.3 (2005), pp. 379–384.
- [30] Imran Ahmed and Gwanggil Jeon. “Enabling artificial intelligence for genome sequence analysis of COVID-19 and alike viruses”. In: *Interdisciplinary sciences: computational life sciences* 14.2 (2022), pp. 504–519.
- [31] Camille Moeckel et al. “A survey of k-mer methods and applications in bioinformatics”. In: *Computational and Structural Biotechnology Journal* (2024).
- [32] Diego Simón, Juan Cristina, and Héctor Musto. “Nucleotide composition and codon usage across viruses and their respective hosts”. In: *Frontiers in Microbiology* 12 (2021), p. 646300.
- [33] Genshiro Esumi. “The Gene’s GC Content Is the Greatest Source of Inter-Species Differences in Protein Amino Acid Composition”. In: (2025).

Author contributions

A.M.F. and M.S.H. collected the data. A.M.F. implemented the proposed approach and wrote the main manuscript text. M.S.H., M.S.M., and W.I.A. reviewed the manuscript. All authors discussed the results and approved the manuscript.

Appendix

Table 13: Per-class performance metrics for different models using FCGR encoding.

Model	Class	Accuracy	Precision	Recall	Specificity	F1-Score
KNN	1a	0.994	0.979	0.991	0.995	0.985
	1b	0.992	0.980	0.980	0.995	0.980
	2a	0.998	0.978	0.976	0.999	0.977
	2b	0.998	0.972	0.984	0.999	0.978
	2c	0.999	0.961	0.972	0.999	0.967
	3a	0.995	0.983	0.990	0.996	0.986
	3b	0.997	0.955	0.958	0.998	0.957
	4	0.995	0.989	0.973	0.998	0.981
	5	0.998	0.977	0.969	0.999	0.973
	6	0.996	0.991	0.971	0.999	0.981
Random Forest	1a	0.995	0.987	0.990	0.997	0.988
	1b	0.989	0.957	0.988	0.990	0.972
	2a	0.997	0.985	0.959	0.999	0.972
	2b	0.998	0.987	0.961	0.999	0.974
	2c	0.998	0.994	0.911	1.000	0.950
	3a	0.995	0.980	0.991	0.995	0.986
	3b	0.997	0.982	0.920	0.999	0.950
	4	0.992	0.973	0.964	0.996	0.968
	5	0.999	0.988	0.961	1.000	0.975
	6	0.994	0.977	0.970	0.997	0.974
XGBoost	1a	0.994	0.982	0.990	0.996	0.986
	1b	0.991	0.970	0.983	0.993	0.977
	2a	0.997	0.988	0.959	0.999	0.973
	2b	0.997	0.971	0.963	0.999	0.967
	2c	0.998	0.994	0.927	1.000	0.960
	3a	0.995	0.983	0.989	0.996	0.986
	3b	0.997	0.971	0.924	0.999	0.947
	4	0.992	0.967	0.968	0.995	0.967
	5	0.999	1.000	0.961	1.000	0.980
	6	0.993	0.969	0.972	0.996	0.971
MLP	1a	0.798	0.091	0.089	0.887	0.090
	1b	0.827	0.132	0.113	0.912	0.122
	2a	0.817	0.108	0.125	0.891	0.116
	2b	0.839	0.159	0.139	0.918	0.148
	2c	0.818	0.095	0.103	0.895	0.099
	3a	0.839	0.100	0.124	0.902	0.111
	3b	0.815	0.139	0.140	0.896	0.140
	4	0.807	0.104	0.086	0.903	0.094
	5	0.832	0.140	0.146	0.905	0.143
	6	0.844	0.121	0.126	0.912	0.124

Table 14: Training and inference time (in seconds) and memory usage (in MiB) for the different models using FCGR encoding.

Model	Train Time	Infer. Time	Train Memory	Infer. Memory
KNN	0.35	54.93	5902.06	7696.55
Random Forest	24.02	0.91	5657.09	5657.09
XGBoost	89.41	0.72	5664.92	5665.06
MLP	1.50	0.70	53.14	54.73

Table 15: Per-class evaluation metrics for the different models using Label encoding.

Model	Class	Accuracy	Precision	Recall	Specificity	F1-score
KNN	1a	0.985	0.949	0.979	0.987	0.964
	1b	0.981	0.936	0.965	0.985	0.950
	2a	0.987	0.857	0.869	0.993	0.863
	2b	0.991	0.925	0.872	0.997	0.898
	2c	0.995	0.892	0.877	0.998	0.885
	3a	0.964	0.895	0.918	0.975	0.907
	3b	0.987	0.826	0.789	0.994	0.807
	4	0.973	0.915	0.872	0.988	0.893
	5	0.997	0.960	0.931	0.999	0.945
	6	0.978	0.931	0.882	0.991	0.906
Random Forest	1a	0.995	0.992	0.984	0.998	0.988
	1b	0.993	0.986	0.979	0.997	0.982
	2a	0.995	0.974	0.925	0.999	0.949
	2b	0.996	0.986	0.914	0.999	0.949
	2c	0.998	0.982	0.933	1.000	0.957
	3a	0.973	0.898	0.969	0.974	0.932
	3b	0.995	0.980	0.858	0.999	0.915
	4	0.981	0.918	0.937	0.988	0.927
	5	0.998	0.988	0.946	1.000	0.966
	6	0.988	0.959	0.938	0.994	0.948
XGBoost	1a	0.995	0.991	0.984	0.998	0.988
	1b	0.992	0.979	0.980	0.995	0.979
	2a	0.996	0.975	0.932	0.999	0.953
	2b	0.996	0.978	0.919	0.999	0.948
	2c	0.998	0.977	0.933	1.000	0.954
	3a	0.977	0.921	0.964	0.981	0.942
	3b	0.995	0.969	0.872	0.999	0.918
	4	0.980	0.909	0.933	0.986	0.921
	5	0.998	0.988	0.946	1.000	0.966
	6	0.985	0.939	0.936	0.992	0.938
MLP	1a	0.810	0.101	0.090	0.900	0.095
	1b	0.791	0.097	0.093	0.884	0.095
	2a	0.820	0.093	0.089	0.902	0.091
	2b	0.824	0.031	0.035	0.898	0.033
	2c	0.835	0.053	0.062	0.903	0.057
	3a	0.801	0.074	0.075	0.888	0.074
	3b	0.836	0.077	0.105	0.896	0.089
	4	0.820	0.143	0.127	0.906	0.135
	5	0.810	0.111	0.113	0.893	0.112
	6	0.835	0.133	0.106	0.920	0.118

Table 16: Training and inference time (in seconds) and memory usage (in MiB) for different models using Label encoding.

Model	Train Time	Infer. Time	Train Memory	Infer. Memory
KNN	0.908	89.537	2791.68	4824.87
Random Forest	59.665	1.783	2864.32	2864.35
XGBoost	154.957	0.672	3145.83	2867.16
MLP	1.501	0.701	53.04	54.57

Table 17: Per-class performance metrics for different models using One-hot encoding.

Model	Class	Accuracy	Precision	Recall	Specificity	F1-Score
KNN	1a	0.974	0.904	0.973	0.974	0.937
	1b	0.970	0.890	0.961	0.973	0.924
	2a	0.987	0.875	0.845	0.994	0.860
	2b	0.993	0.931	0.911	0.997	0.921
	2c	0.997	0.947	0.894	0.999	0.920
	3a	0.973	0.933	0.922	0.985	0.927
	3b	0.991	0.902	0.827	0.997	0.863
	4	0.979	0.953	0.879	0.994	0.914
	5	0.994	0.941	0.861	0.998	0.899
	6	0.981	0.954	0.882	0.994	0.917
Random Forest	1a	0.994	0.991	0.980	0.998	0.986
	1b	0.990	0.973	0.974	0.994	0.974
	2a	0.995	0.974	0.913	0.999	0.943
	2b	0.997	0.981	0.943	0.999	0.961
	2c	0.998	0.988	0.922	1.000	0.954
	3a	0.973	0.896	0.970	0.974	0.931
	3b	0.994	0.962	0.869	0.999	0.913
	4	0.981	0.912	0.942	0.987	0.927
	5	0.996	0.979	0.896	0.999	0.935
	6	0.986	0.963	0.915	0.995	0.938
XGBoost	1a	0.994	0.990	0.983	0.997	0.986
	1b	0.988	0.965	0.972	0.992	0.969
	2a	0.994	0.969	0.906	0.999	0.936
	2b	0.997	0.984	0.945	0.999	0.964
	2c	0.998	0.982	0.927	1.000	0.954
	3a	0.975	0.906	0.968	0.977	0.936
	3b	0.994	0.954	0.865	0.999	0.907
	4	0.982	0.922	0.939	0.988	0.931
	5	0.997	0.987	0.896	1.000	0.939
	6	0.985	0.952	0.924	0.994	0.938
MLP	1a	0.840	0.184	0.152	0.921	0.167
	1b	0.817	0.120	0.112	0.902	0.116
	2a	0.828	0.083	0.087	0.903	0.085
	2b	0.827	0.172	0.143	0.913	0.156
	2c	0.825	0.140	0.172	0.892	0.155
	3a	0.815	0.098	0.081	0.907	0.089
	3b	0.837	0.129	0.128	0.911	0.128
	4	0.823	0.111	0.129	0.894	0.119
	5	0.811	0.105	0.143	0.878	0.121
	6	0.827	0.118	0.108	0.909	0.113

Table 18: Training and inference time (in seconds) and memory usage (in MiB) for different models using One-hot encoding.

Model	Train Time (s)	Infer. Time (s)	Train Memory (MiB)	Infer. Memory (MiB)
KNN	0.3843	52.5305	6533.01	8605.93
Random Forest	17.5565	0.8052	6763.37	6665.28
XGBoost	97.2310	0.7462	6951.26	6634.51
MLP	1.5007	0.7007	52.91	54.43